thws Technische Hochschule Würzburg-Schweinfurt

Projektarbeit

INTERAKTIVE LERNPLATTFORM FÜR GRUNDLAGEN INFORMATIK

M. Nöth, R. Schröder, E. Ebert, D. T. Pham, K. Popov

thws
Projektarbeit
INTERAKTIVE LERNPLATTFORM FÜR GRUNDLAGEN INFORMATIK
M. Nöth, R. Schröder, E. Ebert, D. T. Pham, K. Popov

Person: Enrico
Zeit: ca. 0,5 min

- Begrüßung + Titel nennen
- Team + Betreuer kurz vorstellen
- Agenda skizzieren: Kontext → Lösung → Fachmodule → KI → QS → Demo → Ausblick

1. KONTEXT UND ZIELE

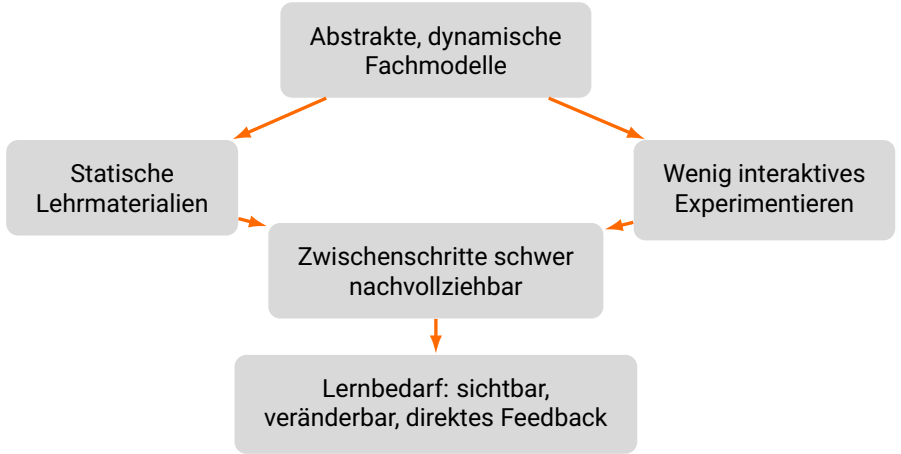
2026-09-07

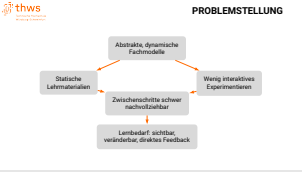
Interaktive LernplattformfürGrundlagen Informatik

Person: Enrico

thws
Technische Hochschule
Würzburg-Schweinfurt

PROBLEMSTELLUNG





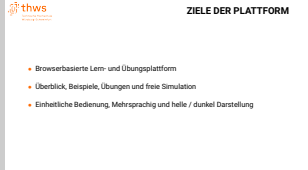
Person: Enrico
Zeit: 4 min

- GI-Themen sind abstrakt + dynamisch (TM, Automaten, DNF, Zahlensysteme)
- Lernende müssen Abläufe mental nachvollziehen
- Typische Hürden:
  - Zustandsübergänge / Bandbewegung
  - Logische Umformungen ohne Visualisierung
  - IEEE 754 / Rundungsfehler schwer greifbar
- Statische Folien = kein Experimentieren, kein Feedback
- Lösung: Plattform mit Visualisierung, Schritt-Simulation, sofortigem Feedback

thws
Technische Hochschule
Würzburg-Schweinfurt

ZIELE DER PLATTFORM

- Browserbasierte Lern- und Übungsplattform
- Überblick, Beispiele, Übungen und freie Simulation
- Einheitliche Bedienung, Mehrsprachig und helle / dunkel Darstellung



Person: Tan
Zeit: 2 min

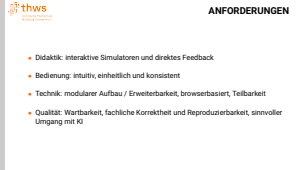
- **Ziel:** Browserbasierte Lern- und Übungsplattform
- **Zugang:** Keine Installation, kein Account, kein Backend
- **Vorbereitung**: ohne lange Vorbereitung genutzt; einfachen Einstieg
- **Vier Bereiche:** Überblick, Beispiele, Übungen und freie Simulation
- **Lernweg:** Nachvollziehen, selbst lösen und ausprobieren
- **Ergebnis:** Entstehung der Ergebnisse sichtbar machen
- **Orientierung:** Einheitliche Bedienung, passende Fachdarstellung
- **Oberfläche:** Mehrsprachig sowie hell und dunkel

2. ANFORDERUNGEN UND PROJEKTABLAUF

thws
Technische Hochschule
Würzburg-Schweinfurt

ANFORDERUNGEN

- Didaktik: interaktive Simulatoren und direktes Feedback
- Bedienung: intuitiv, einheitlich und konsistent
- Technik: modularer Aufbau / Erweiterbarkeit, browserbasiert, Teilbarkeit
- Qualität: Wartbarkeit, fachliche Korrektheit und Reproduzierbarkeit, sinnvoller Umgang mit KI

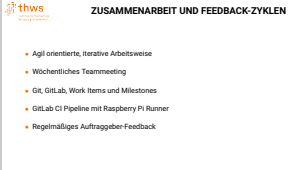


Person: Markus
Zeit: 1,5 min

- **Didaktik:** Abstrakte GI-Themen (Turing, Automaten, DNF, Zahlensysteme) interaktiv visualisieren; Schritt-Simulation und sofortiges Feedback statt statischer Folien.
- **Bedienung:** Konsistentes UI/UX (Design-System, einheitliche Navigation, i18n); keine Hürden für Erstsemester (keine Accounts/Installation nötig).
- **Technik:** 100% Client-Side SPA im Browser; modular erweiterbar für neue Fächer; Teilbarkeit von Aufgaben und Lösungen via Transferstrings.
- **Qualität & KI:** Mathematisch/formale Korrektheit nach Vorlesungsnotation; reproduzierbare Tests (Vitest, Playwright); KI gezielt für Entwicklung/Tests genutzt.

ZUSAMMENARBEIT UND FEEDBACK-ZYKLEN

- Agil orientierte, iterative Arbeitsweise
- Wöchentliches Teammeeting
- Git, GitLab, Work Items und Milestones
- GitLab CI Pipeline mit Raspberry Pi Runner
- Regelmäßiges Auftraggeber-Feedback



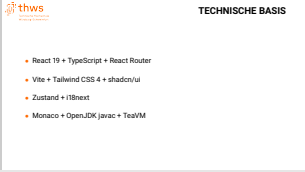
Person: Markus
Zeit: 1,5 min

- **Agil & Iterativ:** Schnelle, lauffähige Prototypen der Kernsimulatoren statt langer isolierter Vorplanung.
- **Wöchentliche Meetings:** Fester Jour Fixe zur Abstimmung plattformweiter Standards (Transfer-Interface, UI-Komponenten) und Blocker.
- **GitLab-Organisation:** Transparente Aufgabenverteilung über Work Items, Feature-Branches und Meilensteine.
- **CI-Pipeline:** Automatisierte Gates (Lint, Typecheck, 1.100+ Tests) sichern Codequalität vor jedem Merge.
- **Auftraggeber-Feedback:** Kontinuierlicher Abgleich mit Dozenten zur Sicherung von Vorlesungskonformität und didaktischem Mehrwert.

3. LÖSUNGSANSATZ: ARCHITEKTUR UND DESIGN

TECHNISCHE BASIS

- React 19 + TypeScript + React Router
- Vite + Tailwind CSS 4 + shadcn/ui
- Zustand + i18next
- Monaco + OpenJDK javac + TeaVM



Person: Tan
Zeit: 0,5 min

- **Frontend:** React 19 + TypeScript + React Router
- **Build und Design:** Vite + Tailwind CSS 4 + shadcn/ui
- **Zustand und Sprache:** Zustand + i18next
- **Java:** Monaco + OpenJDK javac + TeaVM

MODULARE PLATTFORMARCHITEKTUR

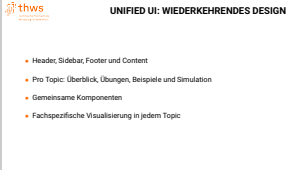
- Fachlich getrennte Topics
- Gemeinsame Shared Components
- `moduleCatalog.tsx` als zentrale Modulverwaltung

Person: Tan
Zeit: 1 min

- **Topics:** Eigene Fachlogik und Ansichten, unabhängig entwickeln
- **Shared Components:** Wiederkehrende Teile, weniger doppelter Code
- **Modulkatalog:** Zentrale Liste in `moduleCatalog.tsx`
- **Einträge:** Pfad, Bezeichnung, Icon und Seite → Navigation und Routen
- **Erweiterung:** Neue Topics zentral ergänzen

UNIFIED UI: WIEDERKEHRENDES DESIGN

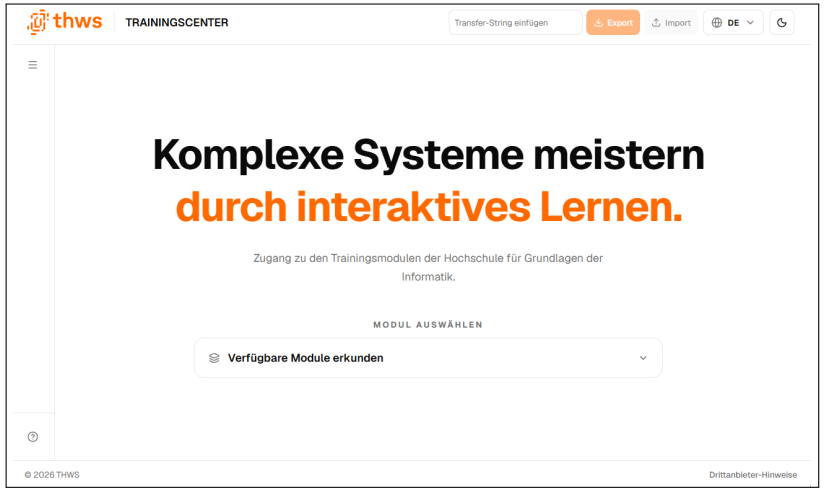
- Header, Sidebar, Footer und Content
- Pro Topic: Überblick, Übungen, Beispiele und Simulation
- Gemeinsame Komponenten
- Fachspezifische Visualisierung in jedem Topic



Person: Tan
Zeit: 1 min

- **Rahmen:** Header, Sidebar, Footer und Inhalt
- **Bereiche:** Immer gleich angeordnet
- **Gemeinsame Komponenten:** Einheitliches Aussehen (Eingaben, Schaltflächen, Rückmeldung)
- **Visualisierung:** jedes Topic eigenes fachliche Design (Tabellen, Zustandgraphen, schrittweise Formelherleitung)

UI – MAINPAGE

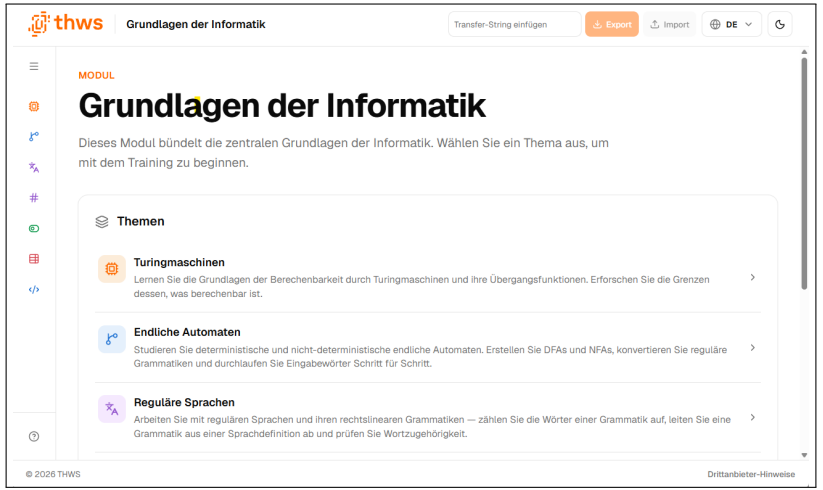




Person: Tan Zeit: 10s

- **Mainpage:** Hauptseite mit den verschiedenen Lernbereichen

UI – MODULPAGE

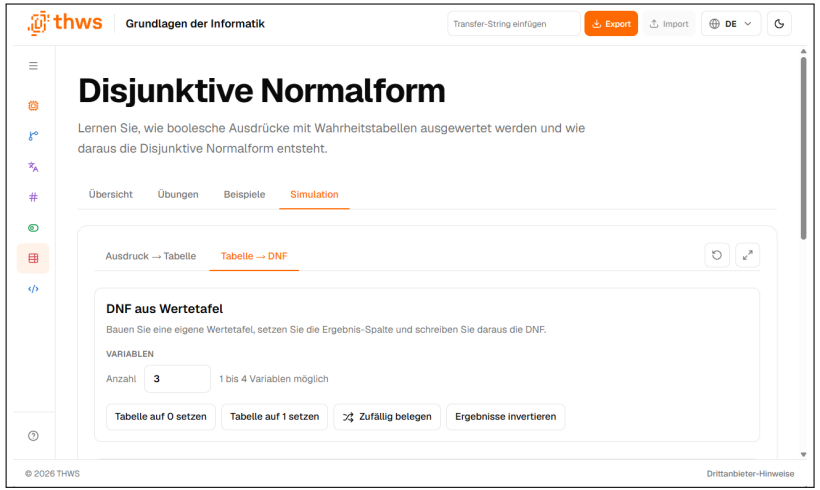


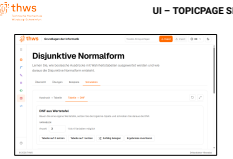


Person: Tan Zeit: 10s

- **Modulpage:** Topics anzeigen und auswählen

UI – TOPICPAGE SIMULATOR

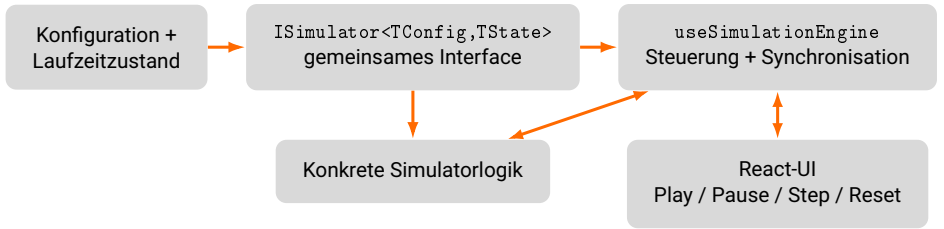


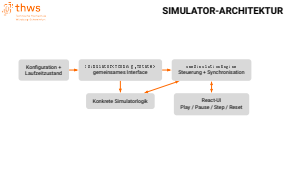


Person: Tan Zeit: 10s

- **Topicpage:** Gemeinsames Design erhalten, fachliche Darstellung

SIMULATOR-ARCHITEKTUR





Person: Richard
Zeit: 2,5 min

- **Architekturprinzip:** Strikte Trennung von formaler Rechenlogik und UI-Rendering. Fachlogik ist reines TypeScript per Vitest unit-testbar.
- **Gemeinsames Interface:** `ISimulator` für alle schrittbasierten Modelle (TM, EA, MM). Methoden: `initialize`, `step`, `stepBack`, `reset`, `getCurrentState`, `isFinished`.
- **Konkrete Simulatoren** Kapseln mathematische Übergangsfunktion (δ) und Validierung.
- **Steuerungs-Hook** `useSimulationEngine`**:** Schlägt die Brücke zur reaktiven React-Welt. Kapselt Browser-Timer (`setInterval`), Ausführungsstatus (`IDLE`, `RUNNING`, `PAUSED`, `ERROR`), Geschwindigkeit (`speed`) und stoppt automatisch bei `isFinished`.
- **Wiederverwendbare UI (**`SimulatorControls`**):** Standardisierte Bedienelemente (Play, Pause, Step vor/zurück, Reset, Tempo-Slider) für alle Simulatoren.

IMPORT, EXPORT UND LOKALE PERSISTENZ

- Einheitliches Transfer-Codec Interface
- Kompakte Base64URL-Transferstrings (12Bit topicStorageId + 6 Bit taskStorageId + Payload)
- Kollisionsfreie lokale Persistenz



Person: Markus
Zeit: 1,5 min

- **Interface:** `ISimulatorTransferCodec<TValue>` statt Monolith. Inversion of Control: Simulator meldet Export beim Mount im Zustand-Store an; Header bleibt entkoppelt.
- **Base64URL:** 3 Zeichen Header (2 Chars Topic-ID, 1 Char Task-ID) fürs Routing. URL-sicher nach RFC 4648 (`-/_`, kein Padding `=`). BitWriter für Automaten, Tokenizer + Deflate für Java.
- **Local Storage:** Zentraler Key `"TrainingCenterStorage"`. Kollisionsfrei durch 18-Bit Topic- und Task-Namespacing. Entprelltes Autosave, autom. Restore beim Laden.

4. KI-EINSATZ IM PROJEKT

KI-WERKZEUGE IM PROJEKT

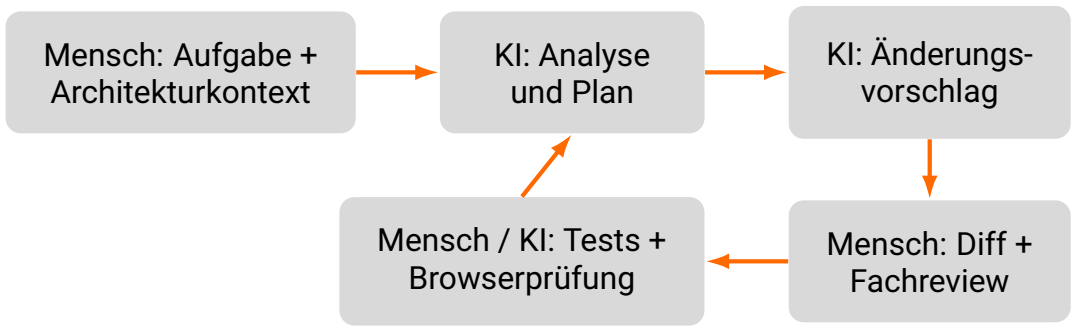
- Stitch, ChatGPT, Gemini, Claude, Codex, Antigravity, verschiedene OpenCode und lokale Modelle
- MCP, Graphify und Agent Skills

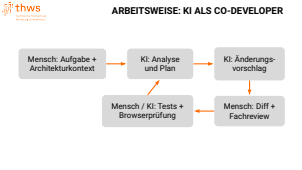


Person: Kristian
Zeit: 1 min

- Jeder von uns hat dabei eigene Tools ausprobiert und Erfahrungen gesammelt
- **Stitch**: Frontend – UI-Ideen und erste Oberflächen
- **ChatGPT, Gemini, Claude, Codex, Antigravity**: als Coding-Agents eingesetzt
- verschiedene **OpenCode** und **lokale Open-Source-Modelle** über Ollama ausprobiert
- Tools und Plugins mit der KI verbunden, um ihre Fähigkeiten zu erweitern
- **MCP**: KI kann Browser & Developer Tools zugreifen und die App direkt testen
- **Graphify**: konnte unsere Projektstruktur als Wissensgraph darstellen, keine Neuanalyse pro Session → spart Tokens und Zeit
- **Agent Skills**: konnten wir der KI zeigen, wie sie bestimmte Aufgaben erledigen soll

ARBEITSWEISE: KI ALS CO-DEVELOPER



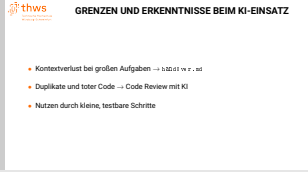


Person: Kristian
Zeit: 2 min

- **Ablauf** nicht vorgegeben, sondern bei allen 5 unabhängig entwickelt
- **Aufgabe + Architekturkontext**: Hier beschreibe ich die Aufgabe detailliert und erkläre, wo im Projekt die Änderung hingehört. BSP: Sie passt eine Shared-Komponente an, obwohl eine lokale Anpassung im Modul richtig wäre
- **Plan vor Code**: Sie liest sich in die betroffenen Dateien ein und beschreibt, was sie ändern will, bevor eine Zeile Code entsteht. Erkennen wir früh Missverständnisse.
- **Diff + Fachreview**: Ich lese den Diff und prüfe, ob das Ergebnis fachlich und didaktisch stimmt. Für den Review habe ich bewusst einen neuen Chat geöffnet
- **Tests**: manuell (Layout im Browser) und von der KI automatisch (Vitest, Playwright)
- **Pfeil zurück**: Finden wir noch Fehler, geht das Feedback direkt zurück an die KI und die Korrekturschleife beginnt von vorn

GRENZEN UND ERKENNTNISSE BEIM KI-EINSATZ

- Kontextverlust bei großen Aufgaben → `handover.md`
- Duplikate und toter Code → Code Review mit KI
- Nutzen durch kleine, testbare Schritte



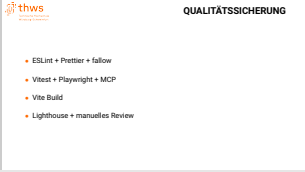
Person: Richard
Zeit: 1,5 min

- **Kontextverlust:** LLMs verlieren bei längeren Sessions den Faden. Die Datei `handover.md` hält offene Aufgaben und Architektur-Entscheidungen verbindlich fest.
- **Code-Qualität:** Modelle bauen gerne eigene Hilfsfunktionen, obwohl es sie im Projekt schon gibt. Manuelle Diff-Reviews und gezielte Refactoring-Prompts verhindern das.
- **Arbeitsweise:** Ein großer Prompt auf einmal funktioniert fast nie. Zuverlässige Ergebnisse gab es nur in kleinen Schritten von der Schnittstelle bis zu den Tests.
- **Dokumentation:** Der Dozent wollte viel KI-Nutzung sehen. Wir haben damit eigene Stichpunkte ausformuliert und Texte geglättet, aber jeden Satz selbst geprüft.

5. QUALITÄT UND DEPLOYMENT-STRATEGIE

QUALITÄTSSICHERUNG

- ESLint + Prettier + fallow
- Vitest + Playwright + MCP
- Vite Build
- Lighthouse + manuelles Review

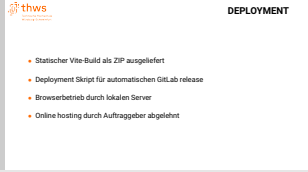


Person: Tan
Zeit: 1 min

- **ESLint + Prettier:** Code prüfen, Format einheitlich halten
- **fallow:** Toten Code, Duplikate und komplexe Stellen finden
- **Vitest:** Komponenten und Fachlogik testen
- **Playwright + MCP:** Vollständige Browserabläufe prüfen
- **Vite-Build:** Erfolgreiche Kompilierung absichern
- **Lighthouse + Review:** Performance, Barrierefreiheit und fachliche Entscheidungen kontrollieren

DEPLOYMENT

- Statischer Vite-Build als ZIP ausgeliefert
- Deployment Skript für automatischen GitLab release
- Browserbetrieb durch lokalen Server
- Online hosting durch Auftraggeber abgelehnt



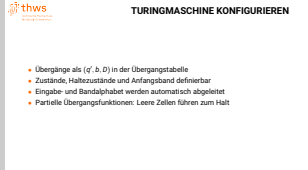
Person: Markus
Zeit: 1 min

- **Vite-Build:** Vollständig statische Client-SPA (`dist/`) als ZIP; benötigt keinerlei Server-Runtime oder Datenbank.
- **GitLab-Release:** CI-Job baut, testet und paketiert bei Tags vollautomatisch; erzeugt versionierte GitLab-Releases.
- **Lokaler Server vs. file://:** `file://` blockiert ES-Module, dyn. Chunks und Monaco-Web-Worker (CORS/Origin null). Lokaler Mini-Server liefert zwingend benötigten HTTP-Origin.
- **Hosting vs. PWA:** Kein zentrales Web-Hosting (keine Serverkosten/Wartung/DSGVO). PWA scheitert offline ohne initiale HTTPS-Domain; lokale ZIP-Verteilung 100% autark.

FACHMODUL: TURINGMASCHINE

TURINGMASCHINE KONFIGURIEREN

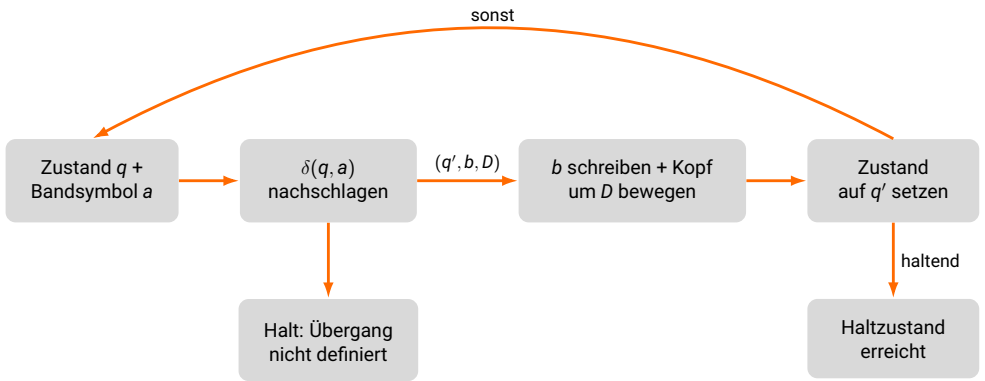
- Übergänge als (q', b, D) in der Übergangstabelle
- Zustände, Haltezustände und Anfangsband definierbar
- Eingabe- und Bandalphabet werden automatisch abgeleitet
- Partielle Übergangsfunktionen: Leere Zellen führen zum Halt

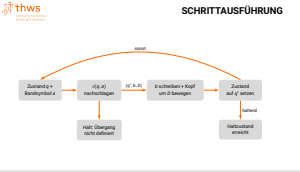


Person: Richard
Zeit: 1 min

- **Tabelle & Tripel** (q', b, D)**:** Zeilen = Bandsymbole ($\diamond$ für Blank, Eingabe <>), Spalten = Nicht-Haltezustände. Jede Zelle definiert Zielzustand q', Schreibsymbol b und Kopfbewegung $D \in \{L, R, N\}$.
- **Editor & Integrität:** q_0, Haltezustände F und Startband frei konfigurierbar. Kaskadierender Reducer passt bei Umbenennung oder Löschung alle Zellen und Zustände atomar an.
- **Automatische Alphabete:** Σ aus Startband abgeleitet, Γ ergänzt $\diamond$ sowie Tabellensymbole. Verhindert Redundanz und Inkonsistenzen.
- **Partielle Übergänge:** Leere Zellen führen regulär zum Halt (`missing_transition`). Erlaubt schrittweises Testen unfertiger Maschinen.

SCHRITTAUSFÜHRUNG





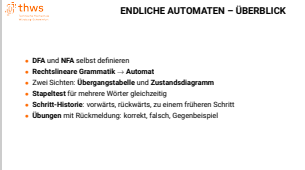
Person: Richard
Zeit: 1 min

- **Lookup & Band:** Liest Symbol an Kopfposition (Default $\diamond$). Schlägt $\delta(q, a)$ in Tabelle nach; virtuelles Band expandiert dynamisch (`Record<number, TapeSymbol>`).
- **Schrittausführung:** Schreibt b, bewegt Kopf ($D \in \{-1, 0, +1\}$), wechselt Zustand auf q' und inkrementiert `stepCount`.
- **Zwei Haltepfade:** Zelle leer $\rightarrow$ `missing_transition`. Haltezustand erreicht ($q' \in F$) $\rightarrow$ `halt_state`. Reine Datenzustände ohne Exceptions.
- **UI-Synchronisation:** Hebt aktive Tabellenzelle $\delta(q, a)$ hervor; Bandbewegung wird geglättet animiert (120–600 ms).

FACHMODUL: ENDLICHE AUTOMATEN

ENDLICHE AUTOMATEN – ÜBERBLICK

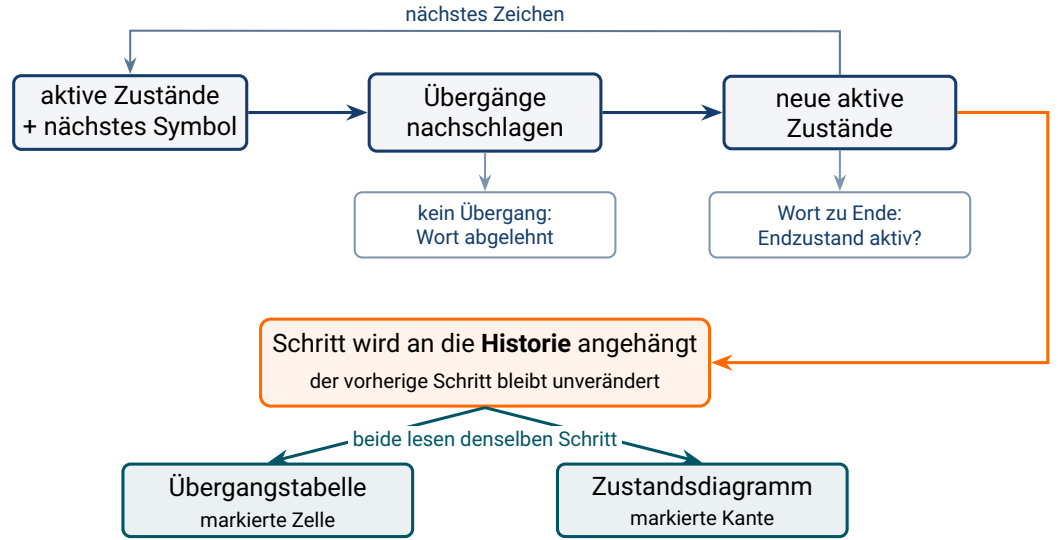
- **DFA** und **NFA** selbst definieren
- **Rechtslineare Grammatik** → **Automat**
- Zwei Sichten: **Übergangstabelle** und **Zustandsdiagramm**
- **Stapeltest** für mehrere Wörter gleichzeitig
- **Schritt-Historie**: vorwärts, rückwärts, zu einem früheren Schritt
- **Übungen** mit Rückmeldung: korrekt, falsch, Gegenbeispiel

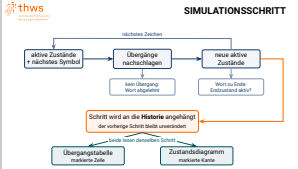


Person: Kristian
Zeit: 1,5 min

- Automat selbst definieren: Typ, Alphabet, Zustände, Startzustand, Endzustände, Übergänge und testen ihn anschließend mit einem Testwort.
- **Grammatik** → **Automat**: Man kann eine RL Grammatik eingeben und.... Jedes Nichtterminal wird Zustand, jede Produktion wird Übergang
- **Zwei Sichten**: Tabelle und Diagramm zeigen denselben Schritt, immer synchron
- **Stapeltest**: Prüft mehrere Wörter auf einmal.
- **Schritt-Historie**: Vorwärts rückwärts oder zu einem bestimmten Schritt springen.
- **Übungen**: Übungen werden automatisch korrigiert. Dabei prüft das System das Verhalten des Automaten und bei Fehlern zeigt direkt das erste falsche Wort als Gegenbeispiel an.

SIMULATIONSSCHRITT





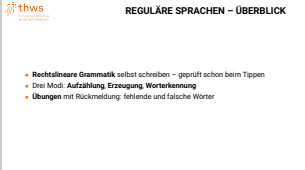
Person: Kristian
Zeit: 1 min

- **Aktive Zustände + nächstes Symbol** → passende Übergänge suchen.
- Daraus entstehen die neuen aktiven Zustände
- Nächstes Zeichen beginnt der Ablauf wieder von vorne.
- Zwei Ausgänge: kein Übergang → abgelehnt. Wortende → Endzustand entscheidet.
- **Historie**: jeder Schritt gespeichert → vor- und zurückspringen.
- Tabelle und Diagramm zeigen denselben Schritt, immer synchron.

FACHMODUL: REGULÄRE SPRACHEN

REGULÄRE SPRACHEN – ÜBERBLICK

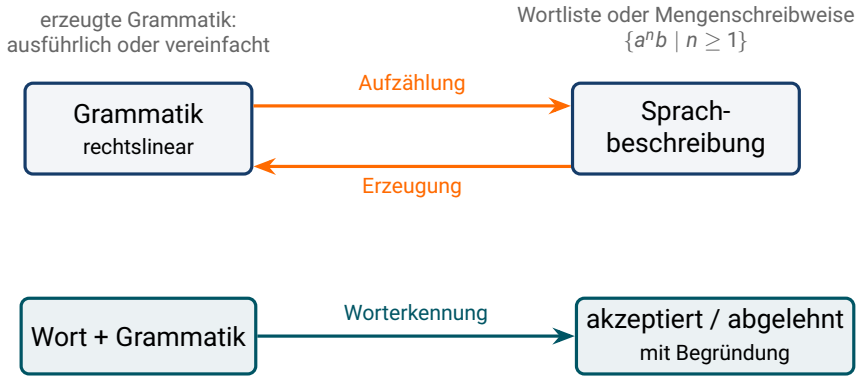
- **Rechtslineare Grammatik** selbst schreiben – geprüft schon beim Tippen
- Drei Modi: **Aufzählung**, **Erzeugung**, **Worterkennung**
- **Übungen** mit Rückmeldung: fehlende und falsche Wörter

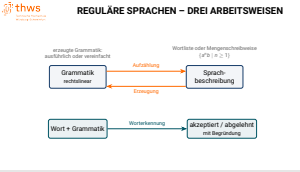


Person: Kristian
Zeit: 1 min

- Die Grammatik wird Zeile für Zeile eingegeben und live beim Tippen geprüft. Ein Berechnen-Knopf fehlt bewusst: Jeder Fehler – wie eine falsche Produktion oder ein nicht deklariertes Symbol – wird direkt in der betroffenen Zeile gemeldet.
- Drei Modi nur benennen → Details auf der nächsten Folie.
- **Übungen**: Prüfung der erkannten Sprache durch konkrete Gegenbeispiele wie fehlende oder falsche Wörter. Genau wie bei den Automaten

REGULÄRE SPRACHEN – DREI ARBEITSWEISEN





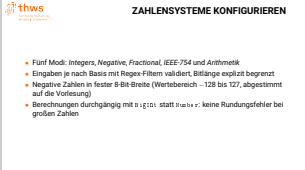
Person: Kristian
Zeit: 1 min

- **Aufzählung**: Grammatik → Wörter der Sprache. Man kann nach Länge und Anzahl begrenzen.
- **Erzeugung**: Die Gegenrichtung. Sprache → Grammatik. Sprache kann man als **Wortliste** oder **Mengenschreibweise** darstellen.
- **Ausführlich** → jedes Präfix ein eigenes Nichtterminal bekommt.
- **Vereinfachte** Grammatik → Präfixe werden zusammengefasst.
- **Worterkennung**: Mehrere Wörter gegen die Grammatik testen. Ergebnis: Man bekommt Begründung statt nur ja/nein.

FACHMODUL: ZAHLENSYSTEME

ZAHLENSYSTEME KONFIGURIEREN

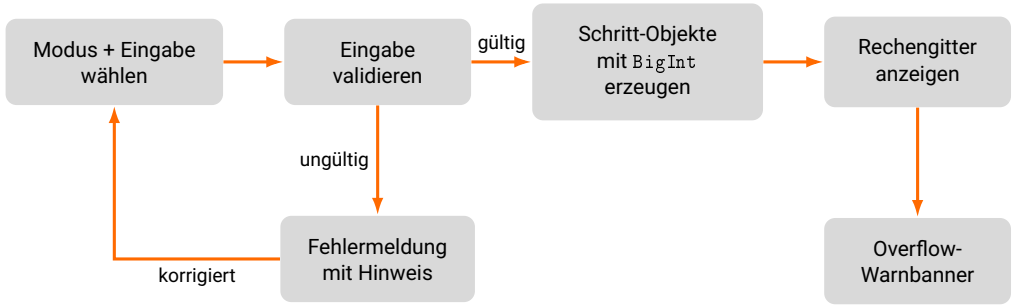
- Fünf Modi: *Integers*, *Negative*, *Fractional*, *IEEE-754* und *Arithmetik*
- Eingaben je nach Basis mit Regex-Filtern validiert, Bitlänge explizit begrenzt
- Negative Zahlen in fester 8-Bit-Breite (Wertebereich −128 bis 127, abgestimmt auf die Vorlesung)
- Berechnungen durchgängig mit `BigInt` statt `Number`: keine Rundungsfehler bei großen Zahlen

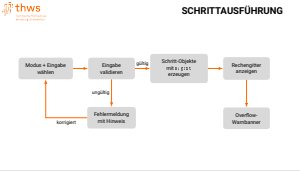


Person: Enrico
Zeit: 1,5 min

- Deckt Vorlesungsstoff ab: Basisumrechnung bis IEEE 754
- 5 Modi: Integers, Negative, Fractional, IEEE-754, Arithmetik
- Eingaben basisgerecht validiert, Bitlänge begrenzt
- Negative: 8 Bit, Wertebereich -128 bis 127 (Vorlesung)
- Berechnungen mit `BigInt` statt `Number` – keine Rundungsfehler
- Überleitung: Ablauf in der App zeigen

SCHRITTAUSFÜHRUNG





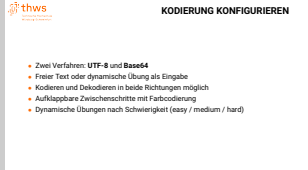
Person: Enrico
Zeit: 1 min

- Ablauf in der App, nicht das Rechenverfahren
- Modus + Eingabe → validieren → Schritt-Objekte → Rechengitter
- Ungültige Eingabe = Hinweis, Nutzer korrigiert
- Stufen aufklappbar, i18n-Texte bleiben beim Sprachwechsel erhalten
- Overflow-Banner warnt, korrigiert nicht – didaktisch bewusst
- Darstellung passt sich Modus an:
  - Wandler: klassische Mathematik-Blöcke
  - Arithmetik: farbcodiertes Rechengitter
  - Negative: Vorzeichen, Einer-, Zweierkomplement
  - IEEE-754: Sign / Exponent / Mantisse getrennt

FACHMODUL: KODIERUNG

KODIERUNG KONFIGURIEREN

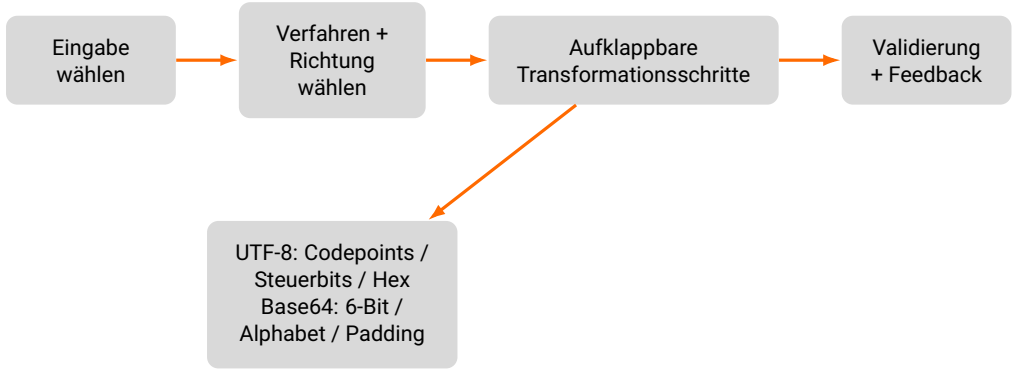
- Zwei Verfahren: **UTF-8** und **Base64**
- Freier Text oder dynamische Übung als Eingabe
- Kodieren und Dekodieren in beide Richtungen möglich
- Aufklappbare Zwischenschritte mit Farbcodierung
- Dynamische Übungen nach Schwierigkeit (easy / medium / hard)



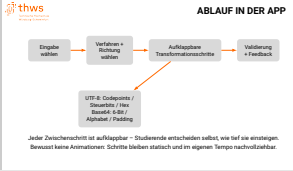
Person: Enrico
Zeit: 1 min

- Modul kam auf Wunsch des Auftraggebers zusätzlich hinzu
- Eingabe wählbar: freier Text oder Übung
- Verfahren + Richtung (kodieren / dekodieren) wählbar
- Details aufklappbar: UTF-8 (Codepoints, Steuerbits, Hex) oder Base64 (6-Bit, Alphabet, Padding)
- Farbcodierung hilft, Bit-Muster zu erkennen
- Übungen dynamisch generiert, nicht persistent – bewusst als Wegwerf-Tests
- Freier Simulator speichert dagegen automatisch
- Überleitung: "Wie läuft das konkret in der App ab?"

ABLAUF IN DER APP



Jeder Zwischenschritt ist aufklappbar – Studierende entscheiden selbst, wie tief sie einsteigen.
Bewusst keine Animationen: Schritte bleiben statisch und im eigenen Tempo nachvollziehbar.



Person: Enrico
Zeit: 1 min

- Ablauf in der App, nicht nur der Algorithmus
- Eingabe: freier Text oder Übungsaufgabe
- Verfahren wählen: UTF-8 oder Base64; Richtung: kodieren / dekodieren
- Darstellung: aufklappbare Stufen mit Farbcodierung (z. B. UTF-8 Steuerbits rot, Nutzbits blau)
- Details je nach Verfahren:
  - UTF-8: Codepoints, Steuerbits, Hex-Darstellung
  - Base64: 8-Bit → 6-Bit, Alphabet, Padding
- Keine Animationen – statisch, eigenes Tempo, besser für Verständnis
- Validierungspanel gibt Lösung erst bei korrekter Eingabe frei

FACHMODUL: DISJUNKTIVE NORMALFORM

DISJUNKTIVE NORMALFORM – ÜBERBLICK

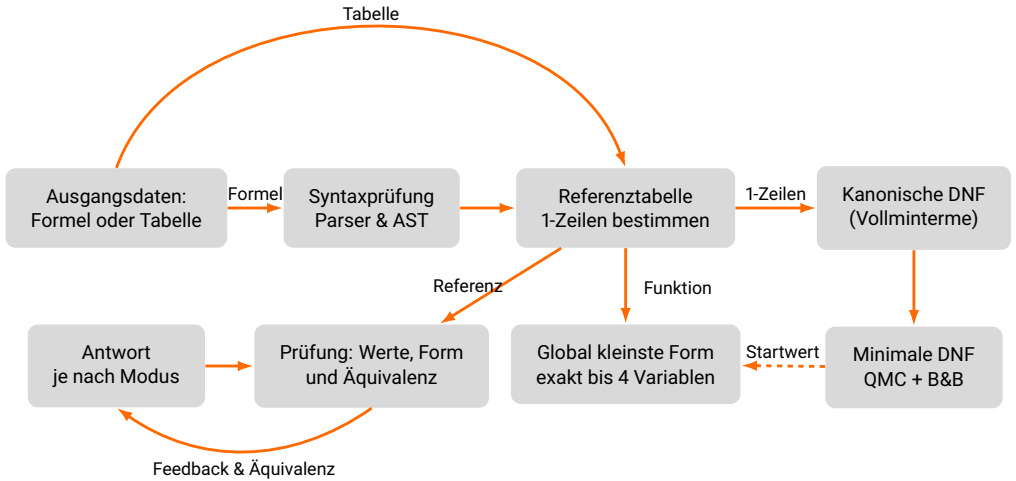
- Zwei Aufgabenmodi: Ausdruck → Wahrheitstabelle und Wahrheitstabelle → DNF
- Drei Ergebnisstufen: kanonische DNF, minimale DNF und optional global kleinste UND/ODER-Formel
- Minimierung mit Quine-McCluskey und Branch-and-Bound
- Semantische Prüfung erkennt logische Äquivalenz
- Begrenzung auf maximal 4 Variablen für exakte Synthese

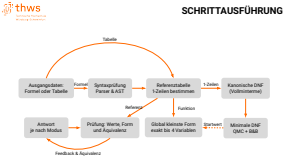
thws
DISJUNKTIVE NORMALFORM – ÜBERBLICK
- Zwei Aufgabenmodi: Ausdruck → Wahrheitstabelle und Wahrheitstabelle → DNF
- Drei Ergebnisstufen: kanonische DNF, minimale DNF und optional global kleinste UND/ODER-Formel
- Minimierung mit Quine-McCluskey und Branch-and-Bound
- Semantische Prüfung erkennt logische Äquivalenz
- Begrenzung auf maximal 4 Variablen für exakte Synthese

Person: Tan
Zeit: 1 min

- **Aufbau:** UND-Terme werden durch ODER verbunden
- **Kanonisch:** Jede 1-Zeile ergibt einen Vollminterm
- **Minimal:** Weniger Literale, exakt mit QMC + B&B
- **Global:** Ausklammern erlaubt, daher nicht zwingend DNF
- **Äquivalenz:** Funktion über Wahrheitstabelle vergleichen
- **Grenze:** Bis vier Variablen garantiert optimal

SCHRITTAUSFÜHRUNG



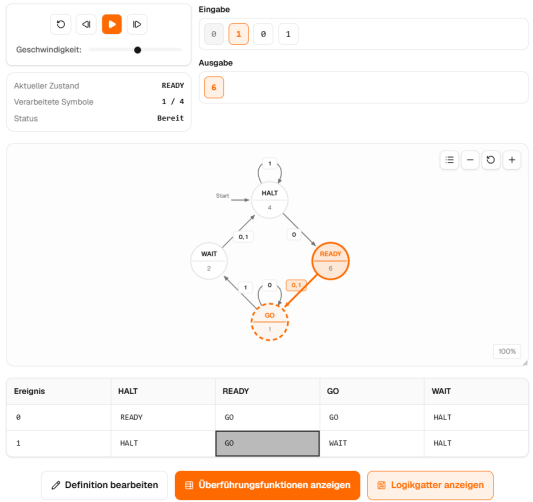


Person: Tan
Zeit: 1,5 min

- **Referenz:** Wahrheitstabelle der erwarteten Funktion
- **Eingabewege:** Formel parsen, Tabelle direkt verwenden
- **Kanonisch:** 1-Zeilen werden zu Vollmintermen
- **Minimal:** Primimplikanten bilden und exakt auswählen
- **Global:** Minimale DNF dient nur als Startwert
- **Freigabe:** Vereinfachen erst nach korrekter kanonischer DNF
- **Feedback:** Werte, Form und Äquivalenz prüfen

FACHMODUL: MOORE MEALY

INTERAKTIVE SIMULATOR DARSTELLUNG



- Automaten frei konfigurieren
- Eingaben Schritt für Schritt simulierbar
- Aktueller Zustand und Übergang direkt sichtbar
- Vor- und Zurückspringen in der Simulation



Person: Markus
Zeit: 1 min

- **Freie Konfiguration:** Zustände und Übergänge interaktiv modellieren; Umschaltung zwischen Moore- und Mealy-Automat.
- **Schritt-Simulation:** Zeichenweise Abarbeitung des Eingabeworts mit visueller Kanten- und Zustandshervorhebung.
- **Zustand & Ausgabe:** Direkte Ausgabeanzeige: bei Moore am Zustand verankert, bei Mealy an der Kante.
- **Trace-Historie:** Vor- und Zurückspringen im Simulations-Stack zum gezielten Auffinden von Denkfehlern.

thws
Technische Hochschule
Würzburg-Schweinfurt

AUTOMAT ANALYSIEREN & SCHALTUNG DARSTELLEN

Automat & Übergänge

- Zustände und Übergänge definieren
- Ausgabe abhängig von Moore bzw. Mealy

Wahrheitstabelle

- Zustände und Eingaben binär codieren
- Boolesche Funktionen automatisch ableiten
- DNF minimieren

Logikgatter

- Funktionen als Schaltung visualisieren
- Gatterdarstellung
- Verbindung von Automat und Hardware

→ **logische bzw. hardwareseitige Umsetzung sichtbar machen**

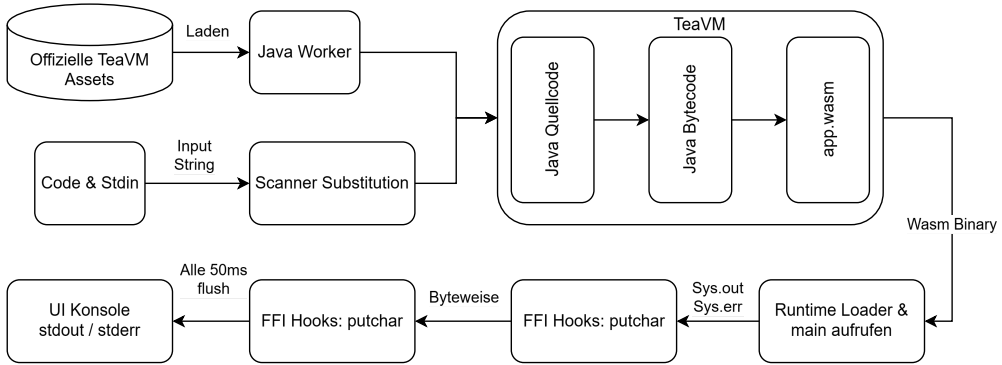


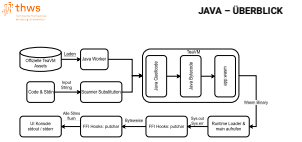
Person: Markus
Zeit: 1 min

- **Automat & Übergänge:** Modellierung des Automaten mit Zustandsübergängen (δ) und Ausgabe (λ) nach Vorlesungsstandard.
- **Wahrheitstabelle:** Binäre Codierung der Zustände ($\lceil \log_2 n \rceil$ Flip-Flop-Bits); automatische Ableitung und Minimierung der DNF.
- **Logikgatter:** Automatische Generierung des Schaltplans (Gatter + D-Flip-Flops); didaktische Brücke zur Digitaltechnik.

FACHMODUL: JAVA

JAVA – ÜBERBLICK



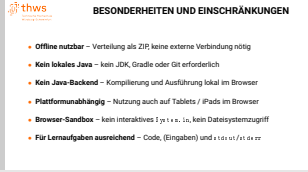


Person: Markus
Zeit: 1 min

- **Client-Architektur:** 100% lokale Ausführung im Browser-Web-Worker; benötigt keinen Backend-Server.
- **Scanner-Substitution:** Vorbereitete Eingaben (`stdin`) werden vorab injiziert, da Web-Worker nicht synchron auf Tastatur blockieren können.
- **TeaVM Pipeline:** Java-Quellcode wird zu Bytecode kompiliert und via TeaVM in WebAssembly (`app.wasm`) übersetzt.
- **FFI & Konsole:** Byteweiser Hook von `putchar` fängt `stdout/stderr` ab und flusht alle 50 ms in die UI.

BESONDERHEITEN UND EINSCHRÄNKUNGEN

- **Offline nutzbar** – Verteilung als ZIP, keine externe Verbindung nötig
- **Kein lokales Java** – kein JDK, Gradle oder Git erforderlich
- **Kein Java-Backend** – Kompilierung und Ausführung lokal im Browser
- **Plattformunabhängig** – Nutzung auch auf Tablets / iPads im Browser
- **Browser-Sandbox** – kein interaktives `System.in`, kein Dateisystemzugriff
- **Für Lernaufgaben ausreichend** – Code, (Eingaben) und `stdout/stderr`



Person: Markus
Zeit: 0.75 min

- **Zero-Installation:** Kein JDK, kein Gradle, keine Pfade; läuft direkt im Browser (auch auf iPads/Tablets).
- **Kein Server:** 100% offlinefähig im ZIP-Paket; keine Serverkosten, kein DSGVO-Risiko.
- **Sandbox & Didaktik:** Sicherheitsisolierung (kein Dateisystem/Netzwerk) schützt den Rechner; optimal zugeschnitten auf GI-Programmieraufgaben.

6. INTERAKTIVE LIVE-DEMO

DEMO-ZIEL: DEN LERNWEG SICHTBAR MACHEN

- Eingabe verändern
- Schrittweise ausführen
- Zwischenschritte erklären

thws
DEMO-ZIEL: DEN LERNWEG SICHTBAR MACHEN
- Eingabe verändern
- Schrittweise ausführen
- Zwischenschritte erklären

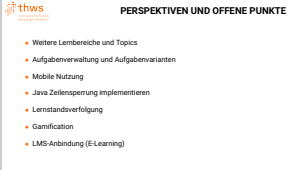
Person:
Zeit:

- test
- test
- test

7. AUSBLICK

PERSPEKTIVEN UND OFFENE PUNKTE

- Weitere Lernbereiche und Topics
- Aufgabenverwaltung und Aufgabenvarianten
- Mobile Nutzung
- Java Zeilensperrung implementieren
- Lernstandsverfolgung
- Gamification
- LMS-Anbindung (E-Learning)



Person: Kristian
Zeit: 1 min

- **Lernbereiche**: Weitere Themen von den anderen Professoren
- **Aufgaben über die Oberfläche anlegen**: Lehrende könnten Aufgaben, automatische Varianten aus Vorlagen, Quizfragen, Lernkarten und Theoriefragen direkt erstellen.
- **Mobile Nutzung verbessern**: Auf kleinen Bildschirmen oder Tablets.
- **Java**: Zeilensperrung implementieren. Das man nicht die Zeilen ändern kann, die nicht bearbeitet werden sollen.
- **Moodle Verbindung** + Gamification.